



Cyberscope

A *TAC Security* Company

Audit Report **smartvault.ai**

July 2026

Contracts name

SmartVaultToken.sol, BotLocking.sol, Vesting.sol, IVesting.sol

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Source Files	4
Overview	5
BotLocking	5
Vesting	5
SmartVaultToken	5
IVesting	5
Findings Breakdown	6
Diagnostics	7
IPD - Insolvent Payout Design	9
Description	9
Recommendation	9
URA - Uncapped Reward Accrual	10
Description	10
Recommendation	10
TSI - Tokens Sufficiency Insurance	11
Description	11
Recommendation	11
VCFS - Vesting Cap Fails Silently	12
Description	12
Recommendation	13
AFT - Activation Fee Truncation	14
Description	14
Recommendation	14
AME - Address Manipulation Exploit	15
Description	15
Recommendation	15
LRCU - Long Referral Cycles Undetected	16
Description	16
Recommendation	17
NSTBA - Non Standard Tokens Break Accounting	18
Description	18
Recommendation	18
POWD - Permissionless One Way Deposit	19
Description	19
Recommendation	19
RIAFP - Referrer Ignored After First Plan	20

Description	20
Recommendation	20
RCIL - Reward Claims Ignore Lock	21
Description	21
Recommendation	21
RFCN - Reward Fund Check Narrow	22
Description	22
Recommendation	22
SCA - SVT Conversion Assumptions	23
Description	23
Recommendation	23
SDF - Sybil Downline Farming	24
Description	24
Recommendation	24
UDO - Unsupported Decimal Ordering	25
Description	25
Recommendation	25
YRSO - Yearly Return Slightly Off	26
Description	26
Recommendation	26
ZAVR - Zero Amount Vesting Release	27
Description	27
Recommendation	27
L19 - Stable Compiler Version	28
Description	28
Recommendation	28
Functions Analysis	29
Inheritance Graph	32
Flow Graph	33
Summary	34
Disclaimer	35
About Cyberscope	36

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Initial Audit

06 Jul 2026

Source Files

Filename	SHA256
Vesting.sol	24e7ca680f8484f10ec7a35535224316f995239ef002cdbd2b521ff9ce356e91
SmartVaultToken.sol	f0bf926198ab651cac25c0dc52aafd4d0e3ae402e32c53d8b689348063b4e53d
BotLocking.sol	1ffd23ab3a1783238835da3a09d59c6d83096fde992c33481bf6073b091ede17
interfaces/IVesting.sol	699203f9b8c4c9ce45199e752a5967aa58980237a173186c095a7282c2e0cea5

Overview

The BotLocking system is a four contract suite built around a locked deposit investment product, a 7 level referral program, and a token bonus paid out through vesting.

BotLocking.sol is the economic core, Vesting.sol releases the SVT bonuses over one year,

IVesting.sol is the interface between the two, and SmartVaultToken.sol is the token itself.

The design aims to combine fixed term locked deposits, monthly returns, multi-level referral earnings, and a vesting token bonus in one on chain product.

BotLocking

BotLocking.sol holds the main investment lifecycle and almost all of the business logic.

Users pay a fee to activate one of four plans, name a referrer on first activation, then invest.

Half of every payment goes to the provider wallet and half stays in the contract's pool.

Positions earn a fixed monthly return over the lock period, and users can claim rewards early or withdraw principal and reward once the lock ends. Each payout takes a fee split between the developer wallet and the referral pool, which credits uplines across seven levels anything undistributed goes to the treasury. The contract also grants SVT bonuses through the vesting contract and exposes read helpers for plans, positions, and referral data.

Vesting

Vesting.sol is the payout layer for the SVT bonuses earned in BotLocking. Only the validator side can create schedules, and that address is set once by the owner. Each schedule unlocks in a straight line over one year, and users call release themselves to take what has unlocked. All schedules count against one global cap of ten million SVT, after which nothing further can be allocated.

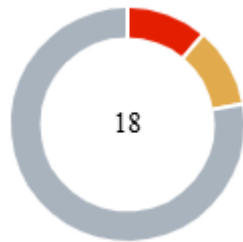
SmartVaultToken

SmartVaultToken.sol is the SVT token and the simplest piece of the system. It is a plain ERC20 with no minting after deploy, no burning, no pausing, no fees, and no blacklist. The full thirty million supply is created once and sent to a treasury address given at deploy.

IVesting

IVesting.sol is a small interface, not a deployed contract. It only lists the two functions BotLocking needs from the vesting side the one that creates a vesting schedule, and the one that reports the SVT token address. It holds no logic, no state, and no funds.

Findings Breakdown



● Critical	2
● Medium	2
● Minor / Informative	14

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	2	0	0	0
● Medium	2	0	0	0
● Minor / Informative	14	0	0	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	IPD	Insolvent Payout Design	Unresolved
●	URA	Uncapped Reward Accrual	Unresolved
●	TSI	Tokens Sufficiency Insurance	Unresolved
●	VCFS	Vesting Cap Fails Silently	Unresolved
●	AFT	Activation Fee Truncation	Unresolved
●	AME	Address Manipulation Exploit	Unresolved
●	LRCU	Long Referral Cycles Undetected	Unresolved
●	NSTBA	Non Standard Tokens Break Accounting	Unresolved
●	POWD	Permissionless One Way Deposit	Unresolved
●	RIAFP	Referrer Ignored After First Plan	Unresolved
●	RCIL	Reward Claims Ignore Lock	Unresolved
●	RFCN	Reward Fund Check Narrow	Unresolved
●	SCA	SVT Conversion Assumptions	Unresolved
●	SDF	Sybil Downline Farming	Unresolved

●	UDO	Unsupported Decimal Ordering	Unresolved
●	YRSO	Yearly Return Slightly Off	Unresolved
●	ZAVR	Zero Amount Vesting Release	Unresolved
●	L19	Stable Compiler Version	Unresolved

IPD - Insolvent Payout Design

Criticality	Critical
Location	BotLocking.sol#L428,505
Status	Unresolved

Description

Every time a user invests, half the money goes straight to the provider wallet and only the other half stays in the contract. But the contract still owes each user 100% of their principal back, plus reward on top. So the pool always holds less than it owes. There is no yield source anywhere rewards and returned principal are paid out of money that later users put in. Early users get paid with the deposits of later users. Once new money stops coming, `_distributeWithdraw` starts reverting on the balance check and everyone still in loses their principal.

```
uint256 providerAmount = amount / 2;  
contractTokenAmount += amount - providerAmount;  
paymentToken.safeTransfer(provider, providerAmount);
```

```
if (contractTokenAmount < amount + reward) revert InsufficientFunds();
```

Recommendation

The team is advised to rethink the whole payout model. If principal must be returned in full, the contract cannot send half of every deposit out the door and expect to stay solvent. Either keep enough of each deposit inside the contract to cover the promised principal and rewards, or back the payouts with a real external yield source. As it stands the system can only pay early users by taking from later ones, and that should be fixed before any launch.

URA - Uncapped Reward Accrual

Criticality	Critical
Location	BotLocking.sol#L526
Status	Unresolved

Description

The reward formula uses the full time since the investment started, with no cap when the lock period ends. Plan 3 pays 10% per month and just keeps going. Leave a position open long enough and the reward owed climbs past 100%, 200%, and beyond, with nothing to stop it. The pool that has to pay this is finite and already under-funded. One patient user can build up a reward claim far larger than anything the contract can cover, and drain it out over time.

```
uint256 secondsSinceStart = currentTimestamp - startTimestamp;  
reward = amount * monthlyReturnBps * secondsSinceStart / (PCT_BASE *  
MONTH_IN_SECONDS);
```

Recommendation

The team should cap reward accrual at the plan's lock duration. Reward should stop growing once the lock ends, so the most a position can ever owe is a known, fixed number. Right now the liability is open ended, which makes the pool impossible to size safely.

TSI - Tokens Sufficiency Insurance

Criticality	Medium
Location	Vesting.sol#L77,108,170 SmartVaultToken.sol#L7
Status	Unresolved

Description

The SVT tokens are not held within the Vesting contract itself. The full supply is minted to a treasury address at deploy, and the contract is designed to receive tokens from that external administrator by hand. It creates schedules and promises releases without ever checking it holds enough SVT to cover them. While external administration provides flexibility, it introduces a dependency on the administrator's actions and a centralization risk if the treasury underfunds the contract, release reverts and users cannot collect a bonus the system already told them they earned. There is also no recovery path for a surplus.

```
constructor(IERC20 _svt) Ownable(msg.sender) {  
    if (address(_svt) == address(0)) revert ZeroAddress();  
    svt = _svt;  
}
```

```
function release(uint256 vestingIndex) public virtual {  
    address beneficiary = msg.sender;  
    uint256 amount = releasable(beneficiary, vestingIndex);  
    vestingSchedules[beneficiary][vestingIndex].released += amount;  
    emit VestingReleased(beneficiary, amount);  
    svt.safeTransfer(beneficiary, amount);  
}
```

Recommendation

It is recommended to consider implementing a more decentralized and automated approach for handling the contract tokens. One possible solution is to hold the tokens within the contract itself. If the contract guarantees the process it can enhance its reliability, security, and participant trust, ultimately leading to a more successful and efficient process.

VCFS - Vesting Cap Fails Silently

Criticality	Medium
Location	Vesting.sol#L205 BotLocking.sol#L588
Status	Unresolved

Description

When the maximum airdrop pool has already been allocated, `_createVestingSchedule()` returns without creating a schedule, reverting, or emitting an event.

```
if (totalDistributedTokens == MAX_AIRDROP_POOL) return;
```

As a result, `BotLocking.invest()` can succeed even though the user or their referrers receive no SVT allocation. The absence of an event or return value prevents the calling contract and affected beneficiaries from determining that the advertised bonus was skipped.

The same problem occurs when the requested allocation exceeds the remaining pool. Instead of rejecting the request, the vesting contract silently reduces the allocation:

```
uint256 amountToAllocate = amount;

if (totalDistributedTokens + amount > MAX_AIRDROP_POOL) {
    amountToAllocate =
        MAX_AIRDROP_POOL - totalDistributedTokens;
}
```

Additionally, `BotLocking` imposes no maximum investment amount. A sufficiently large investment may therefore consume the complete remaining airdrop pool through the investor's direct bonus and first-investment referral bonuses. This makes bonus availability dependent on transaction ordering and may prevent later participants from receiving their expected allocations.

Recommendation

The Team is advised to make insufficient vesting capacity explicit. If the complete calculated bonus is guaranteed to users, schedule creation should revert when the remaining pool cannot satisfy the full allocation.

Alternatively, `createVestingSchedule()` should return the amount actually allocated and emit an event whenever an allocation is partially fulfilled or skipped. BotLocking should then handle this result and clearly communicate it to users.

The Team should also consider limiting investment amounts or reserving SVT allocations in advance to prevent a single investment from unexpectedly consuming the entire remaining pool.

AFT - Activation Fee Truncation

Criticality	Minor / Informative
Location	BotLocking.sol#L72,167
Status	Unresolved

Description

The constructor explicitly narrows each decimal scaled activation fee to uint128, although Plan.activationFee is declared as uint256. If a calculated fee exceeds the uint128 range, the explicit conversion truncates its high-order bits before assigning the result to the wider field. Consequently, a payment token with a sufficiently large decimal count can configure activation fees substantially below their intended values. Common six and 18 decimal tokens are unaffected, but the constructor does not restrict the accepted decimal range.

```
/// @notice One-time activation fee for the plan.
uint256 activationFee;

...
uint256 tokenDecimals = paymentToken.decimals();
_plans[0] = Plan(
    uint128(MONTH_IN_SECONDS),
    200,
    2400,
    uint128(100 * 10 ** tokenDecimals)
);
```

Recommendation

Consider preserving the full width activation fee calculation instead of applying a narrower conversion. If a restricted range is intentional, validate both the token decimal count and the calculated fee during deployment before performing any conversion.

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	BotLocking.sol#L37
Status	Unresolved

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

```
IERC20Metadata public immutable paymentToken;
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

LRCU - Long Referral Cycles Undetected

Criticality	Minor / Informative
Location	BotLocking.sol#L535,550
Status	Unresolved

Description

`_checkRefererCirculation` traverses only `REF_LEVEL_COUNT` referral links, which is fixed at seven. A cycle containing eight or more addresses is therefore accepted because the traversal terminates before following the edge that returns to the initial referrer. Current referral distributions are also limited to seven levels, so such a cycle does not cause repeated payouts, excess distribution, or an infinite loop. The issue is informational because it violates the intended acyclic referral graph invariant and could affect future functionality that assumes the graph contains no cycles.

```
_participants[wallet].referrer = referrer;  
_checkRefererCirculation(referrer);
```

```
function _checkRefererCirculation(address referer) internal view {  
    address directReferer = referer;  
    if (referer != address(0)) {  
        for (uint i = 0; i < REF_LEVEL_COUNT; i++) {  
            referer = _participants[referer].referrer;  
            if (referer == address(0)) break;  
            if (referer == directReferer) revert  
                ReferralCirculationDetected();  
        }  
    }  
}
```

Recommendation

Consider enforcing a maximum referral chain depth of seven levels during registration. A new referral relationship should be rejected if it would extend the chain beyond this limit, ensuring that the stored referral graph remains consistent with the maximum depth used by the referral distribution system.

NSTBA - Non Standard Tokens Break Accounting

Criticality	Minor / Informative
Location	BotLocking.sol#L420,494
Status	Unresolved

Description

The contract credits `contractTokenAmount` using the amount it was told to pull, not the amount it actually received. If the payment token takes a fee on transfer, the contract gets less than it records, so its books show more money than it holds, and later withdrawals fail. A rebasing token causes the same drift. The payment token is fixed at deploy and never checked for this behavior.

```
paymentToken.safeTransferFrom(msg.sender, address(this), amount);
_investments[msg.sender].push(Investment(amount, block.timestamp,
planIndex, 0, false));
_participants[msg.sender].totalLockedAmount += amount;
...
contractTokenAmount += amount - providerAmount;
```

Recommendation

The team is advised to base accounting on the balance actually received measure the contract's balance before and after the transfer and use the difference. Or simply document that only plain, non-fee, non-rebasing tokens are supported and enforce that choice at deploy. Right now a fee on transfer token silently corrupts the records.

POWD - Permissionless One Way Deposit

Criticality	Minor / Informative
Location	BotLocking.sol#L493
Status	Unresolved

Description

Anyone can call deposit to add liquidity, and there is no way to take a bare deposit back out. Money put in this way can only ever leave through user withdrawals. A well meaning depositor effectively subsidizes the payout pool with no claim to their funds.

```
function deposit(uint256 amount) public nonReentrant {  
    paymentToken.safeTransferFrom(msg.sender, address(this), amount);  
    contractTokenAmount += amount;  
    emit Deposited(msg.sender, amount);  
}
```

Recommendation

The team should make clear who is expected to call this and that deposits are not refundable. If only the provider is meant to top up the pool, consider restricting the caller so random users don't lose funds by mistake.

RIAFP - Referrer Ignored After First Plan

Criticality	Minor / Informative
Location	BotLocking.sol#L390
Status	Unresolved

Description

A referrer is only recorded on the very first activation. On any later activation the referrer argument is accepted and thrown away with no error. A user who tries to set or correct their referrer later gets no warning that nothing happened.

```
bool isFirstPlanActivation = true;
for (uint256 i = 0; i < PLAN_COUNT; i++) {
    if (_participants[msg.sender].isActive[i]) {
        isFirstPlanActivation = false;
        break;
    }
}
if (isFirstPlanActivation) {
    _registerReferrer(msg.sender, referrer);
}
```

Recommendation

The team should make this behavior clear to users, either by documenting that the referrer is fixed at first activation or by reverting if a referrer is passed when one is already set. Silently ignoring the input invites confusion.

RCIL - Reward Claims Ignore Lock

Criticality	Minor / Informative
Location	BotLocking.sol#L460
Status	Unresolved

Description

claimRewards lets a user pull accrued reward without ever checking whether the lock period has passed. It only checks the pool has enough for the reward. So a user can activate a plan, invest, and then claim reward again and again from day one, while their principal stays locked and keeps earning. Combined with the uncapped reward and the under funded pool, this is the cheapest way to bleed the contract dry. Whoever claims first gets paid whoever's late gets InsufficientFunds.

```
function claimRewards(uint256 investmentIndex) public nonReentrant {
    if (investmentIndex >= _investments[msg.sender].length) revert
    InvalidInvestmentIndex();
    Investment storage investment =
    _investments[msg.sender][investmentIndex];
    if (investment.isLockedAmountWithdrawn) revert
    InvestmentAlreadyWithdrawn();
    ...
    if (contractTokenAmount < reward) revert InsufficientFunds();
}
```

Recommendation

It is recommended that the team decide clearly whether reward can be claimed mid lock at all. If reward is meant to be paid only at maturity, add the same lock expiry check that withdraw uses. If early claiming is intended, then the reward math must be capped and the pool must actually be funded to cover it, or this becomes a first come drain.

RFCN - Reward Fund Check Narrow

Criticality	Minor / Informative
Location	BotLocking.sol#L474,504
Status	Unresolved

Description

claimRewards checks the pool covers the reward, but the payout also sends the caller's referral balance in the same transfer. The referral part is funded from a separate pool, and the overall balance does cover it, so this doesn't cause a loss today. Still, the check is narrower than the amount actually paid, which is fragile if the accounting ever changes.

```
if (contractTokenAmount < reward) revert InsufficientFunds();  
...  
_distributeWithdraw(msg.sender, 0, reward, referralReward);
```

Recommendation

The team should make the funds check match everything the call pays out, so the guard stays correct even if the pool structure is changed later.

SCA - SVT Conversion Assumptions

Criticality	Minor / Informative
Location	BotLocking.sol#L558
Status	Unresolved

Description

The SVT bonus calculation applies the plan multiplier directly to the payment-token amount and subsequently adjusts only for the difference between the tokens' decimals. Decimal normalization converts base units between the two token formats, but it does not account for their relative economic values. Consequently, the calculation implicitly treats one whole payment token as equivalent to one whole SVT. This may cause users to receive substantially more or less SVT than intended and may also accelerate exhaustion of the fixed vesting pool if SVT is undervalued by the conversion formula.

```
uint256 svtBonus =  
    investmentAmount * svtBonusMultiplier / PCT_BASE;  
  
svtBonus =  
    svtBonus * 10 ** (svtDecimals - paymentTokenDecimals);
```

Recommendation

The Team should clarify whether `svtBonusMultiplier` represents a token-quantity ratio or a percentage of the investment's monetary value.

If it represents a value-based bonus, the calculation should use a configured conversion rate or a manipulation-resistant oracle to convert the payment-token value into the corresponding SVT amount. If the fixed one-to-one token ratio is intentional, this assumption should be explicitly documented so that users understand how bonuses are calculated.

SDF - Sybil Downline Farming

Criticality	Minor / Informative
Location	BotLocking.sol#L483,533
Status	Unresolved

Description

A referrer can be any address. There is no check that the referrer ever joined, invested, or did anything. On top of that, `withdrawReferralReward` lets any address pull its referral balance with no requirement to be active. So an attacker can spin up throwaway wallets, point them all at one master wallet as referrer, and farm referral cuts and referral SVT into a wallet they control. There are no real people in the network just the attacker's own wallets feeding rewards upward. It costs activation fees and locked capital, but if SVT has value the farm pays for itself.

```
function _registerReferrer(address wallet, address referrer) private {
    if (referrer == wallet) revert InvalidReferrer();
    _participants[wallet].referrer = referrer;
    _checkRefererCirculation(referrer);
}
```

```
function withdrawReferralReward() public nonReentrant {
    uint256 referralReward =
    _participants[msg.sender].referralSystemReward;
    if (referralReward == 0) revert NoReferralReward();
}
```

Recommendation

It is recommended that the team require a referrer to be a genuine participant before it can earn for example, someone who has activated a plan and actually invested. This raises the cost of building fake downlines. It won't stop a determined Sybil, but it removes the free version of the attack where empty wallets collect rewards.

UDO - Unsupported Decimal Ordering

Criticality	Minor / Informative
Location	BotLocking.sol#L167, 591
Status	Unresolved

Description

The SVT bonus conversion assumes that SVT always has at least as many decimals as the payment token. If the payment token has more decimals, the subtraction used to calculate the scaling factor underflows and reverts under Solidity's checked arithmetic. Because `_distributeSvtBonus` is executed during every investment, an unsupported decimal ordering prevents all users from investing. No deployment time validation currently enforces the assumed relationship between the two tokens.

```
uint256 tokenDecimals = paymentToken.decimals();  
...  
uint256 svtDecimals = svt.decimals();  
uint256 paymentTokenDecimals = paymentToken.decimals();  
uint256 svtBonus = investmentAmount * svtBonusMultiplier / PCT_BASE;  
svtBonus = svtBonus * 10 ** (svtDecimals - paymentTokenDecimals);
```

Recommendation

Consider supporting conversion in both decimal directions, applying multiplication or division according to the relative token precisions. Alternatively, validate the supported decimal relationship during deployment and reject incompatible token configurations with a clear error.

YRSO - Yearly Return Slightly Off

Criticality	Minor / Informative
Location	BotLocking.sol#L33,168,523
Status	Unresolved

Description

Reward is figured in 30-day months, but the year lock is 365 days. 365 does not divide evenly into 30-day months, so a "10% monthly" plan held for its full one-year lock actually pays about 121.7%, not a clean 120%. It's a small gap, but if the intended headline number was exactly 120% per year, the code doesn't match that intent.

```
uint256 public constant MONTH_IN_SECONDS = 30 days;
uint256 public constant YEAR_IN_SECONDS = 365 days;
...
function _calculatePlanReward(uint256 amount, uint256 planIndex, uint256
startTimestamp) private view returns (uint256 reward) {
    uint256 monthlyReturnBps = _plans[planIndex].monthlyReturnBps;
    uint256 currentTimestamp = block.timestamp;
    uint256 secondsSinceStart = currentTimestamp - startTimestamp;
    reward = amount * monthlyReturnBps * secondsSinceStart / (PCT_BASE *
MONTH_IN_SECONDS);
}
```

Recommendation

The team should confirm whether returns are meant to be exact monthly or exact yearly figures, and pick month and year lengths that line up with the advertised rate so the paid amount matches what users are promised.

ZAVR - Zero Amount Vesting Release

Criticality	Minor / Informative
Location	Vesting.sol#L172
Status	Unresolved

Description

release doesn't check that anything is actually claimable. If called too early or after everything is claimed, it still updates state, emits a release event with zero, and does a zero-value transfer. Nothing is lost, but it produces misleading events and wastes gas.

```
uint256 amount = releasable(beneficiary, vestingIndex);
vestingSchedules[beneficiary][vestingIndex].released += amount;
emit VestingReleased(beneficiary, amount);
svt.safeTransfer(beneficiary, amount);
```

Recommendation

The team is advised to skip or reject a release when the claimable amount is zero, so events only fire on real transfers.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	SmartVaultToken.sol#L2 Vesting.sol#L2 BotLocking.sol#L2 interfaces/IVesting.sol#L2
Status	Unresolved

Description

The ^ symbol indicates that any version of Solidity that is compatible with the specific version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
pragma solidity ^0.8.28;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

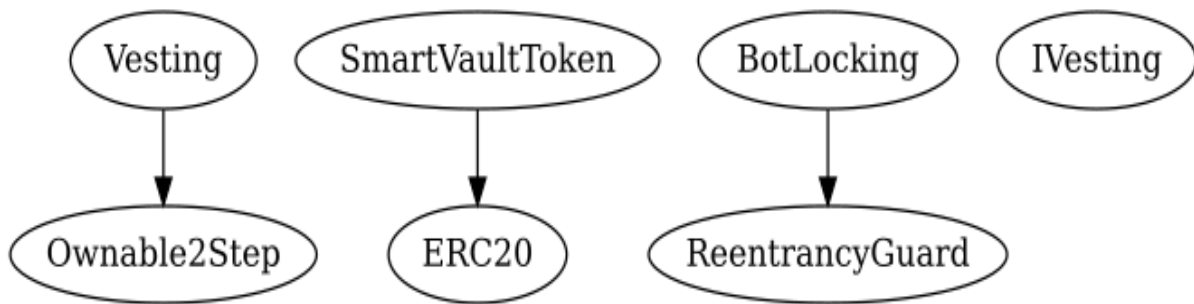
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
Vesting	Implementation	Ownable2Step		
		Public	✓	Ownable
	vestingSchedulesLength	External		-
	setBotLocking	External	✓	onlyOwner
	createVestingSchedule	External	✓	-
	duration	Public		-
	start	Public		-
	end	Public		-
	released	Public		-
	releasable	Public		-
	release	Public	✓	-
	vestedAmount	Public		-
	_createVestingSchedule	Internal	✓	
	_vestingSchedule	Internal		
SmartVaultToken	Implementation	ERC20		
		Public	✓	ERC20

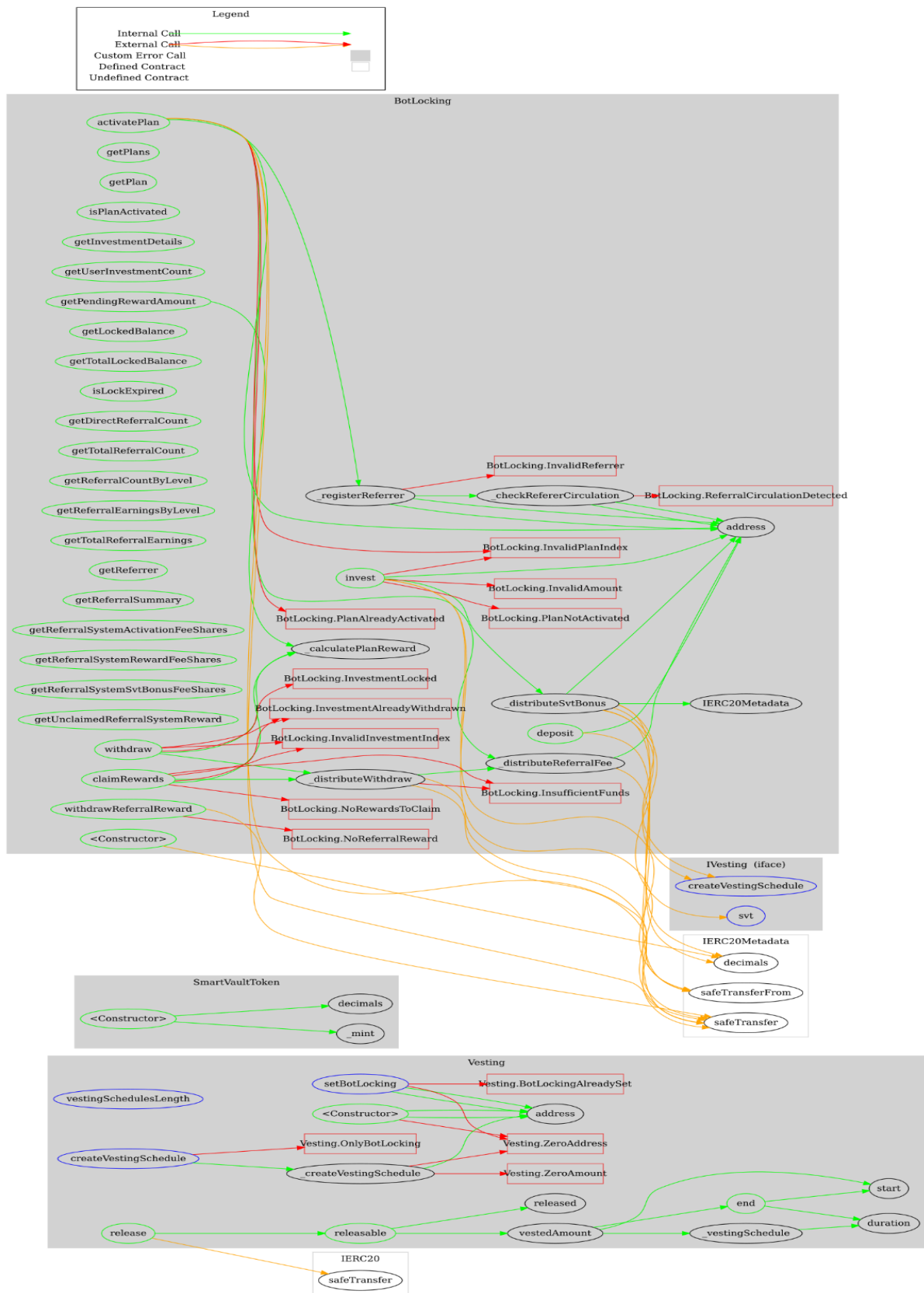
BotLocking	Implementation	ReentrancyGuard		
		Public	✓	-
	getPlans	Public		-
	getPlan	Public		-
	isPlanActivated	Public		-
	getInvestmentDetails	Public		-
	getUserInvestmentCount	Public		-
	getPendingRewardAmount	Public		-
	getLockedBalance	Public		-
	getTotalLockedBalance	Public		-
	isLockExpired	Public		-
	getDirectReferralCount	Public		-
	getTotalReferralCount	Public		-
	getReferralCountByLevel	Public		-
	getReferralEarningsByLevel	Public		-
	getTotalReferralEarnings	Public		-
	getReferrer	Public		-
	getReferralSummary	Public		-
	getReferralSystemActivationFeeShares	Public		-
	getReferralSystemRewardFeeShares	Public		-
	getReferralSystemSvtBonusFeeShares	Public		-
	getUnclaimedReferralSystemReward	Public		-
	activatePlan	Public	✓	nonReentrant

	invest	Public	✓	nonReentrant
	withdraw	Public	✓	nonReentrant
	claimRewards	Public	✓	nonReentrant
	withdrawReferralReward	Public	✓	nonReentrant
	deposit	Public	✓	nonReentrant
	_distributeWithdraw	Private	✓	
	_calculatePlanReward	Private		
	_registerReferrer	Private	✓	
	_checkRefererCirculation	Internal		
	_distributeReferralFee	Private	✓	
	_distributeSvtBonus	Private	✓	
IVesting	Interface			
	createVestingSchedule	External	✓	-
	svt	External		-

Inheritance Graph



Flow Graph



Summary

smartvault.ai contracts implement a locked deposit investment suite built around fixed term plans with monthly returns, a 7 level referral program, and a token bonus layer paid out through vesting. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io